

# 0.26. En diagonale : deplace

`deplace(x, y, tuile, pasX, pasY)` : les deux axes en une ligne. Avec 5 et 5, la lettre descend en biais.

## 1. Ce qu'il faut comprendre

Une seule ligne pour les deux axes : `deplace`.

**Ce qui est nouveau ici** : `deplace(colonne, ligne, tuile, pasX, pasY)`. Elle a **deux** nombres de pas : `pasX` pour l'horizontale (+ droite, - gauche) et `pasY` pour la verticale (+ bas, - haut).

**La diagonale** : avec `5` et `5`, la lettre fait à chaque quart de seconde un pas sur X **et** un pas sur Y **en même temps** : elle descend en biais, (1, 1), (2, 2)... jusqu'à (5, 5). C'est impossible avec `deplace_x` puis `deplace_y`, qui font un axe après l'autre.

**Deux positions à ranger** : une fonction ne rend qu'une valeur. `deplace` rend la colonne, et dépose la ligne dans une variable de la console. **Seule sur sa ligne**, la console range les deux : `x` **et** `y`. Écris-la toujours ainsi : `x = deplace(...)` ne rangerait que `x`.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1806 octets** de cartouche.

### 1. UNE VARIABLE — X

```
uint8_t x = 0;           // la COLONNE de la lettre
```

### 2. UNE VARIABLE — Y

```
uint8_t y = 0;           // la LIGNE de la lettre
```

### 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    //  deplace(x, y, ALPHABET[0], 5, 5);
    //      | | |           | |
    //      | | |           | +-- pasY : +5, vers le bas
    //      | | |           +----- pasX : +5, vers la droite
    //      | | +----- tuile : la lettre A
    //      +---+----- x, y : d'où elle part ; la console y
    //                      range l'arrivée : x = 5, y = 5
    //
    //  A                (0, 0)
    //      ↘
    //      ↘
    //      ↘           à chaque pas : +1 sur X ET +1 sur Y
    //      ↘
    //      ↘
    //  A                (5, 5)
    deplace(x, y, ALPHABET[0], 5, 5);
    nombre(0, 17, x);    // 005
    nombre(4, 17, y);    // 005

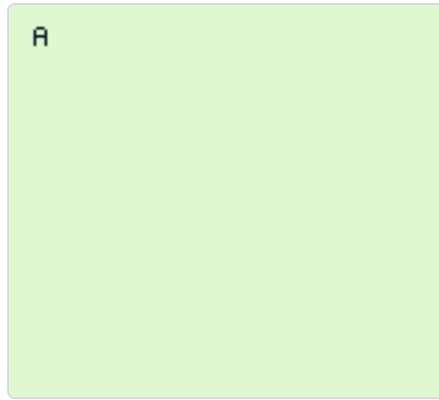
    while (true) {
        image();
    }
}
```

### 3. Ce qui se passe, étape par étape

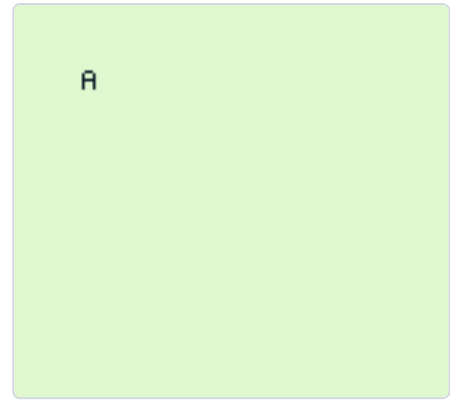
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



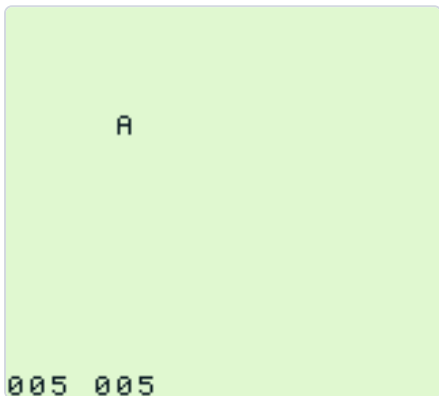
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu'on doit voir

Un A qui descend en diagonale, du coin en haut à gauche jusqu'en (5, 5). En bas, 005 et 005.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en biais : (1, 1), (2, 2), (3, 3), (4, 4)

✓ jamais tout droit : ni (3, 0), ni (0, 3)

✓ x et y rangés tout seuls : 5 et 5

✓ le A est en (5, 5) (5,5)

Ensuite : **0.26.1. En diagonale : deplace — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++